PROGRAMAÇÃO I

ESTRUTURAS DE DADOS EM PYTHON

VOLUME II • CADERNO 04

BUSCAS E ORDENAÇÃO

Cinco algoritmos de ordenação, duas formas de busca e a ideia de dividir um problema em partes.

COMO ESTUDAR Comece pelo problema concreto, acompanhe o estado dos dados antes e depois de cada operação e só então leia o código. Execute os exemplos na ordem. Quando aparecer um termo técnico, ele será definido no próprio capítulo. Não é necessário decorar comandos antes de entender o comportamento.

CONTEXTO DIDÁTICO Os exemplos acompanham uma rede fictícia de lojas, com produtos, clientes, estoque, pedidos e atendimento. Os registros são exemplos de aula, não dados comerciais reais. Este caderno pressupõe os fundamentos de Python e o Caderno 01 (listas).

Série de cadernos independentes e complementares • Texto explicativo, exemplos executáveis e roteiros físicos de representação.

Mapa de aprendizagem deste caderno

Aqui o foco muda: em vez de apenas guardar dados, aprendemos procedimentos para encontrá-los e organizá-los. Cada algoritmo será acompanhado por uma simulação em mesa, um teste de mesa, implementação em Python e critérios para saber quando utilizá-lo. Merge Sort e Quick Sort fazem parte do núcleo obrigatório.

As representações em tabelas mostram o que o computador armazena; as demonstrações físicas mostram como uma pessoa poderia simular as mesmas operações com cartões ou colegas em posições fixas. **O número da posição não se move: o elemento é que muda.**

1. O que significa procurar em uma coleção?

Uma loja pode ter um catálogo com centenas de produtos. Localizar um código é diferente de acessar um índice. O índice informa onde está o elemento; o código identifica qual produto queremos. Quando não sabemos a posição, precisamos de um procedimento de busca.

Ao analisar uma busca, devemos acompanhar três informações: **o alvo** (valor procurado), **o estado da coleção** (se está ordenada ou não) e **as verificações** realizadas. Dois algoritmos podem encontrar exatamente o mesmo produto, mas utilizar quantidades bem diferentes de verificações.

Índice	0	1	2	3	4	5
Código	104	101	106	103	102	105

SIMULAÇÃO Seis alunos seguram cartões com códigos de produtos e ficam em posições fixas numeradas de 0 a 5. Uma pessoa deve encontrar o código 103 sem conhecer seu índice. A turma registra quais posições foram examinadas e em que ordem.

2. Busca linear: verificar um por um

A busca linear começa no primeiro elemento e avança até encontrar o alvo ou até alcançar o fim. Ela funciona mesmo quando os dados estão misturados e não depende de preparação anterior. A principal limitação é que, em um caso desfavorável, pode precisar verificar todos os elementos.

```
def busca_linear(valores, alvo):
    for indice, valor in enumerate(valores):
        if valor == alvo:
            return indice
    return -1

codigos = [104, 101, 106, 103, 102, 105]
print(busca_linear(codigos, 103))  # 3
print(busca_linear(codigos, 999))  # -1
```

Teste de mesa: o alvo é 103 e os códigos estão fora de ordem.

Verificação	Índice	Valor observado	Encontrou 103?
1	0	104	Não
2	1	101	Não
3	2	106	Não
4	3	103	Sim: encerra

O retorno `-1` é uma convenção escolhida por esta função para indicar ausência. Não é um resultado obrigatório de toda busca linear; poderíamos devolver `None`, por exemplo. A decisão deve ser documentada e seguida por quem chama a função.

Buscar registros completos

```
def localizar_por_codigo(produtos, codigo):
    for produto in produtos:
        if produto["codigo"] == codigo:
            return produto
    return None
```

Nesse caso, queremos o **registro**, não o índice. O procedimento é o mesmo, mas o tipo de resposta foi adaptado ao problema. Esse é um bom exemplo de por que compreender um algoritmo vale mais do que decorar uma única implementação.

3. Busca binária: eliminar metade das possibilidades

Se os códigos estiverem **ordenados em ordem crescente**, podemos examinar o elemento central. Quando o alvo é menor que o central, descartamos toda a parte maior; quando é maior, descartamos a parte menor. Repetimos a operação na metade que ainda pode conter o resultado.

Índice	0	1	2	3	4	5	6
Código	101	102	103	104	105	106	107

Passo	Faixa ainda possível	Meio	Comparação
1	0 a 6	índice 3: 104	106 > 104 → procurar à direita
2	4 a 6	índice 5: 106	encontrado

É a mesma estratégia usada quando procuramos uma palavra em um dicionário impresso: não precisamos percorrer todas as páginas anteriores se o material estiver organizado. Em uma lista que não esteja ordenada, descartar metade pode eliminar justamente o item desejado.

```
def busca_binaria(valores, alvo):
    inicio = 0
    fim = len(valores) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        if valores[meio] == alvo:
            return meio
        elif valores[meio] < alvo:
            inicio = meio + 1
        else:
            fim = meio - 1
    return -1

print(busca_binaria([101, 102, 103, 104, 105, 106, 107], 106))
```

A divisão inteira `//` calcula um índice central válido. Quando o meio não contém o alvo, atualizamos os limites para excluir a posição já examinada. O laço termina quando encontramos o valor ou quando `inicio > fim`, indicando que não há posições restantes.

CUIDADO Ordenar os dados tem custo. Se você precisa localizar um único item em uma lista curta e misturada, ordenar tudo antes da busca pode dar mais trabalho do que executar uma busca linear. A busca binária vale especialmente quando a coleção já está ordenada ou haverá muitas consultas.

4. Comparar buscas e compreender a preparação dos dados

Propriedade	Busca linear	Busca binária
Exige ordem?	Não	Sim, pela mesma chave pesquisada
Verificações no pior caso	Até n	Cresce proporcionalmente a $\log_2(n)$
É simples para catálogo pequeno?	Sim	Sim, se a ordem já existir
Devolve índice ou registro?	Depende da implementação	Depende da implementação

Para uma lista de 1.024 elementos, uma busca linear pode examinar 1.024 posições; uma busca binária, com os dados ordenados, precisa de cerca de 11 verificações no pior caso. Isso não significa que o número de segundos será 93 vezes menor: estamos comparando crescimento do trabalho, não cronometrando máquinas.

Buscar nomes exige ordenar por nomes

Se você ordenar os produtos por preço, uma busca binária pelo nome não é válida. A ordem precisa corresponder ao campo que participa da comparação. Em aplicações reais, índices e estruturas especializadas podem permitir pesquisar vários campos sem reorganizar a mesma lista a cada consulta.

5. Por que ordenar? E o que significa uma troca?

Ordenar é reorganizar elementos segundo um **critério**: preço crescente, nome alfabético, estoque decrescente ou data mais recente. A decisão sobre o critério vem antes do algoritmo. Quando dois preços são iguais, podemos preservar a ordem anterior (estabilidade) ou aplicar um desempate explícito.

Nas demonstrações seguintes, utilizaremos os valores `[35, 12, 48, 20, 15]`. Manter a mesma lista facilita comparar procedimentos diferentes sem confundir o aluno com novos dados a cada capítulo.

Índice	0	1	2	3	4
Antes	35	12	48	20	15
Objetivo	12	15	20	35	48

DINÂMICA Cinco voluntários seguram preços e ocupam posições no chão. O professor escolhe uma regra de movimento por vez. Todos observam a sequência após cada operação. Comparar significa apenas examinar valores; trocar significa modificar duas posições.

6. Bubble Sort: comparar vizinhos

O Bubble Sort examina pares de vizinhos. Se o elemento da esquerda é maior do que o da direita, troca os dois. Ao completar uma passagem, o maior elemento da região examinada terá alcançado sua posição final. Passagens seguintes percorrem uma região cada vez menor.

Operação	Sequência resultante	Motivo
Início	35, 12, 48, 20, 15	Desordenada
35 com 12	12, 35, 48, 20, 15	35 > 12: troca
35 com 48	12, 35, 48, 20, 15	35 < 48: mantém
48 com 20	12, 35, 20, 48, 15	48 > 20: troca
48 com 15	12, 35, 20, 15, 48	48 termina na posição final

```
def bubble_sort(valores):
    dados = valores.copy()
    n = len(dados)
    for fim in range(n - 1, 0, -1):
        houve_troca = False
        for i in range(fim):
            if dados[i] > dados[i + 1]:
                dados[i], dados[i + 1] = dados[i + 1], dados[i]
                houve_troca = True
        if not houve_troca:
            break
    return dados

print(bubble_sort([35, 12, 48, 20, 15]))
```

A variável `fim` delimita a região ainda não consolidada. `houve_troca` permite encerrar mais cedo quando toda uma passagem termina sem nenhuma troca: isso indica que os valores já estão em ordem. Para cinco elementos em ordem contrária, serão feitas várias comparações; no pior caso, o trabalho cresce aproximadamente com o quadrado de n.

7. Selection Sort: escolher o menor restante

Diferentemente do Bubble Sort, o Selection Sort procura o menor valor de toda a região não organizada e só então o coloca na primeira posição disponível. A cada rodada, uma posição passa a estar definitivamente organizada.

Rodada	Menor escolhido	Resultado depois da troca
Inicial	—	35, 12, 48, 20, 15
1	12	12, 35, 48, 20, 15
2	15	12, 15, 48, 20, 35
3	20	12, 15, 20, 48, 35
4	35	12, 15, 20, 35, 48

```
def selection_sort(valores):
    dados = valores.copy()
    for inicio in range(len(dados) - 1):
        menor = inicio
        for j in range(inicio + 1, len(dados)):
            if dados[j] < dados[menor]:
                menor = j
        if menor != inicio:
            dados[inicio], dados[menor] = dados[menor], dados[inicio]
    return dados

print(selection_sort([35, 12, 48, 20, 15]))
```

A variável `menor` guarda um **índice**, não o preço. Isso permite saber qual objeto deve trocar de posição ao terminar as comparações. Mesmo que a lista já esteja ordenada, esta versão examina as mesmas combinações de posições; ela pode fazer poucas trocas, mas continua realizando muitas comparações.

COMPARAÇÃO Para seis elementos, a busca pelo menor restante faz 5 + 4 + 3 + 2 + 1 = 15 comparações. Não confunda esse número com a quantidade de trocas: uma rodada pode terminar sem nenhuma troca efetiva.

8. Insertion Sort: encaixar cada elemento no lugar certo

Pense em organizar cartas de baralho na mão. Você mantém uma parte organizada e pega uma nova carta. Enquanto as cartas maiores estiverem à direita da posição correta, você as desloca para abrir espaço e insere a carta escolhida. Esse é o raciocínio do Insertion Sort.

Etapa	Parte já organizada	Sequência completa
Início	35	35, 12, 48, 20, 15
Insere 12	12, 35	12, 35, 48, 20, 15
Insere 48	12, 35, 48	12, 35, 48, 20, 15
Insere 20	12, 20, 35, 48	12, 20, 35, 48, 15
Insere 15	12, 15, 20, 35, 48	12, 15, 20, 35, 48

```
def insertion_sort(valores):
    dados = valores.copy()
    for i in range(1, len(dados)):
        atual = dados[i]
        j = i - 1
        while j >= 0 and dados[j] > atual:
            dados[j + 1] = dados[j]
            j -= 1
        dados[j + 1] = atual
    return dados

print(insertion_sort([35, 12, 48, 20, 15]))
```

Guardar `atual` é necessário porque as posições serão deslocadas. O laço interno recua enquanto encontra valores maiores. Ao terminar, a posição `j + 1` está livre para receber o valor guardado. Em listas quase ordenadas, esse algoritmo costuma realizar poucas movimentações.

9. Recursividade antes dos algoritmos de divisão

Os dois algoritmos seguintes trabalham com uma ideia nova: resolver um problema grande dividindo-o em versões menores. Para expressar isso em Python, uma função pode chamar a si mesma. Esse recurso é a **recursividade**. Toda solução recursiva precisa de uma condição de parada, chamada **caso-base**.

```
def contagem_regressiva(n):
    if n == 0:        # caso-base: interrompe a sequência
        print("Fim")
        return
    print(n)
    contagem_regressiva(n - 1)

contagem_regressiva(3)
```

Chamada	Ação	Próxima chamada
f(3)	mostra 3	f(2)
f(2)	mostra 2	f(1)
f(1)	mostra 1	f(0)
f(0)	mostra Fim	nenhuma

Cada chamada cria um contexto de execução próprio, com seus parâmetros e variáveis locais. Se nunca alcançarmos o caso-base, a função continuará chamando a si mesma até atingir o limite de

recursão. Merge Sort e Quick Sort são bons exemplos de recursão porque cada chamada recebe uma parte menor dos dados.

DINÂMICA Um aluno recebe dez cartões e divide o monte em dois. Cada colega recebe metade e repete a divisão até ficar com uma única carta. Em seguida, as partes são reunidas seguindo uma regra. Separar e depois combinar ajuda a visualizar o Merge Sort; separar em torno de um pivô ajuda a visualizar o Quick Sort.

10. Merge Sort: dividir, ordenar e intercalar

O Merge Sort divide a sequência em duas partes, ordena cada metade e depois **intercala** as duas partes já ordenadas. A intercalação é a etapa central: em vez de procurar livremente entre todos os elementos, comparamos o primeiro valor disponível de cada metade e retiramos o menor.

Fase	Representação
Lista original	[35, 12, 48, 20, 15]
Primeira divisão	[35, 12] | [48, 20, 15]
Metades ordenadas	[12, 35] | [15, 20, 48]
Intercalação	12 → 15 → 20 → 35 → 48
Resultado	[12, 15, 20, 35, 48]

Como intercalar duas listas já ordenadas

```
def intercalar(esquerda, direita):
    resultado = []
    i = j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1
    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])
    return resultado
```

A função usa dois índices independentes. Ela sempre escolhe o menor elemento ainda disponível entre as duas listas. Quando uma das metades termina, adiciona o restante da outra. Usar `<=` preserva a ordem relativa dos elementos iguais provenientes das duas metades, contribuindo para a estabilidade dessa implementação.

Versão completa do Merge Sort

```
def merge_sort(valores):
    if len(valores) <= 1:
        return valores.copy()
    meio = len(valores) // 2
    esquerda = merge_sort(valores[:meio])
    direita = merge_sort(valores[meio:])
    return intercalar(esquerda, direita)

print(merge_sort([35, 12, 48, 20, 15]))
```

O caso-base devolve uma lista vazia ou com um elemento, que já está ordenada. Cada chamada recursiva resolve um pedaço menor. O trabalho de combinar cresce com o tamanho da lista e é

repetido por cerca de $\log_2(n)$ níveis de divisão; daí vem o custo **O(n log n)**. Esta implementação cria novas listas ao dividir e intercalar; por isso consome memória auxiliar proporcional a n, além da pilha de chamadas.

11. Quick Sort: selecionar um pivô e separar grupos

O Quick Sort escolhe um **pivô** e divide os valores em grupos menores, iguais e maiores do que esse pivô. Depois ordena recursivamente os grupos que ainda precisam de organização. Diferentemente do Merge Sort, o trabalho principal ocorre ao **particionar**, não ao intercalar resultados ordenados.

Etapa	Grupo / resultado
Original	[35, 12, 48, 20, 15]
Pivô escolhido	48 (nesta versão, último elemento)
Menores que 48	[35, 12, 20, 15]
Iguais a 48	[48]
Maiores que 48	[]
Ordena recursivamente os menores	[12, 15, 20, 35]
Combinação	[12, 15, 20, 35, 48]

Essa tabela mostra a **ideia** de separação. Para simplificar a primeira implementação, vamos usar três listas. Ela é mais fácil de ler e evita alguns detalhes técnicos de uma versão que realiza todas as trocas dentro da própria lista.

```
def quick_sort(valores):
    if len(valores) <= 1:
        return valores.copy()
    pivo = valores[len(valores) // 2]
    menores = [x for x in valores if x < pivo]
    iguais = [x for x in valores if x == pivo]
    maiores = [x for x in valores if x > pivo]
    return quick_sort(menores) + iguais + quick_sort(maiores)

print(quick_sort([35, 12, 48, 20, 15]))
```

Neste código, o pivô é escolhido pelo índice central, não pelo último valor usado na tabela anterior: a estratégia de escolha pode mudar sem alterar a ideia do algoritmo. As compreensões constroem os três grupos, a função ordena os grupos menores e maiores e concatena os resultados. A lista original não é modificada.

CUIDADO COM A EFICIÊNCIA Quick Sort tem tempo **médio O(n log n)**, mas pode chegar a **$O(n^2)$** quando as divisões ficam muito desequilibradas. A seleção do pivô importa. Esta versão didática de três listas também usa memória adicional; portanto não deve ser confundida com as implementações in-place frequentemente estudadas em cursos avançados.

12. Comparação dos cinco métodos

Algoritmo	Ideia visual	Custo típico / pior caso	Observação
Bubble Sort	Comparar e trocar vizinhos	$O(n^2)$ / $O(n^2)$	Pode encerrar cedo se não houver trocas.
Selection Sort	Escolher o menor restante	$O(n^2)$ / $O(n^2)$	Poucas trocas; muitas comparações.
Insertion Sort	Inserir na parte já ordenada	$O(n^2)$ / $O(n^2)$	Bom em dados quase ordenados.

Merge Sort	Dividir e intercalar	O(n log n) / O(n log n)	Estável nesta implementação; memória adicional.
Quick Sort	Particionar em torno de pivô	Médio O(n log n) / $O(n^2)$	Pivô e balanceamento influenciam o custo.

Tempo de execução não é apenas quantidade de comparações: há movimentações, criação de listas e chamadas recursivas. A notação Big O simplifica a análise do crescimento quando aumentamos os dados. Ela será explicada de modo mais completo no último caderno.

Para uso cotidiano: sort e sorted

```
produtos = [
    {"nome": "Shampoo", "preco": 24.90},
    {"nome": "Perfume", "preco": 129.90},
    {"nome": "Batom", "preco": 29.90}
]
ordenados = sorted(produtos, key=lambda p: p["preco"])
for p in ordenados:
    print(p["nome"], p["preco"])
```

A função `sorted()` retorna uma nova lista. O método `list.sort()` reorganiza a própria lista e devolve `None`. Em Python, essas ferramentas integradas são apropriadas para aplicações comuns e utilizam um algoritmo estável altamente otimizado (Timsort). Implementar Bubble, Selection, Insertion, Merge e Quick continua importante para compreender as estratégias de solução.

13. Uma forma correta de comparar os algoritmos

Para observar diferenças, use exatamente os mesmos dados como entrada de cada função e verifique se todas retornam a mesma lista ordenada. Como as cinco versões didáticas acima retornam uma nova lista, a coleção original pode ser reutilizada nas comparações.

```
entrada = [35, 12, 48, 20, 15]
esperado = sorted(entrada)
for algoritmo in [bubble_sort, selection_sort, insertion_sort,
                  merge_sort, quick_sort]:
    resultado = algoritmo(entrada)
    print(algoritmo.__name__, resultado == esperado)
```

A saída esperada é `True` em todas as linhas. Esse procedimento é um **teste de correção**, não uma medição de desempenho. Para avaliar eficiência, seria necessário também contar operações ou cronometrar várias execuções com tamanhos de dados diferentes, evitando comparar resultados de entradas distintas.

Para guardar deste caderno

- **Busca linear**: encontra valores sem exigir ordem, examinando sequencialmente.
- **Busca binária**: exige ordem e elimina metade das possibilidades a cada passo.
- **Bubble, Selection e Insertion**: três estratégias de ordenação fáceis de simular manualmente.
- **Merge Sort e Quick Sort**: estratégias de divisão do problema; recursividade, caso-base e combinação/particionamento são essenciais.
- Para projetos reais, conheça também `sorted()` e `list.sort()`, sem confundir ferramentas da linguagem com os algoritmos construídos para estudo.

Referências de consulta

Python Software Foundation. The Python Tutorial: Data Structures. https://docs.python.org/3/tutorial/datastructures.html

Python Software Foundation. Biblioteca padrão: collections e json. https://docs.python.org/3/library/collections.html ; https://docs.python.org/3/library/json.html

VisuAlgo. Visualizações e explicações de algoritmos. https://visualgo.net/en

Harvard University. CS50x 2026, Week 3 — Algorithms. https://cs50.harvard.edu/x/weeks/3/

Bhargava, Aditya. Entendendo Algoritmos (Grokking Algorithms), 2ª edição. Manning Publications. Referência de abordagem visual; exemplos e explicações deste caderno são originais.